



STRATEGY PATTERN

Áp dụng trong hệ thống Order Management (C#)



1. Mục tiêu bài học

Sau bài này, sinh viên có thể:

- Hiểu Strategy Pattern là gì, giải quyết vấn đề gì
- Phân biệt Strategy Pattern vs if–else / switch–case
- Thiết kế hành vi thay đổi động tại runtime
- Áp dụng Strategy Pattern vào:
 - Tính phí vận chuyển
 - Tính giảm giá / khuyến mãi
 - Phương thức thanh toán trong Order Management
- Viết code C# đúng chuẩn OCP & DIP



2. Bài toán thực tế trong Order Management

✗ Cách làm sai (code mùi – anti-pattern)

⚠ Vấn đề:

- Vi phạm Open–Closed Principle
- Mỗi lần thêm loại vận chuyển → sửa code cũ
- Code khó test, khó mở rộng
- Dễ sinh lỗi khi hệ thống lớn

```
public decimal CalculateTotal(Order order, string shippingType)
{
    if (shippingType == "Standard")
        return order.Total + 20000;
    else if (shippingType == "Express")
        return order.Total + 50000;
    else if (shippingType == "SameDay")
        return order.Total + 80000;
    throw new Exception("Invalid shipping type");
}
```



3. Strategy Pattern là gì?

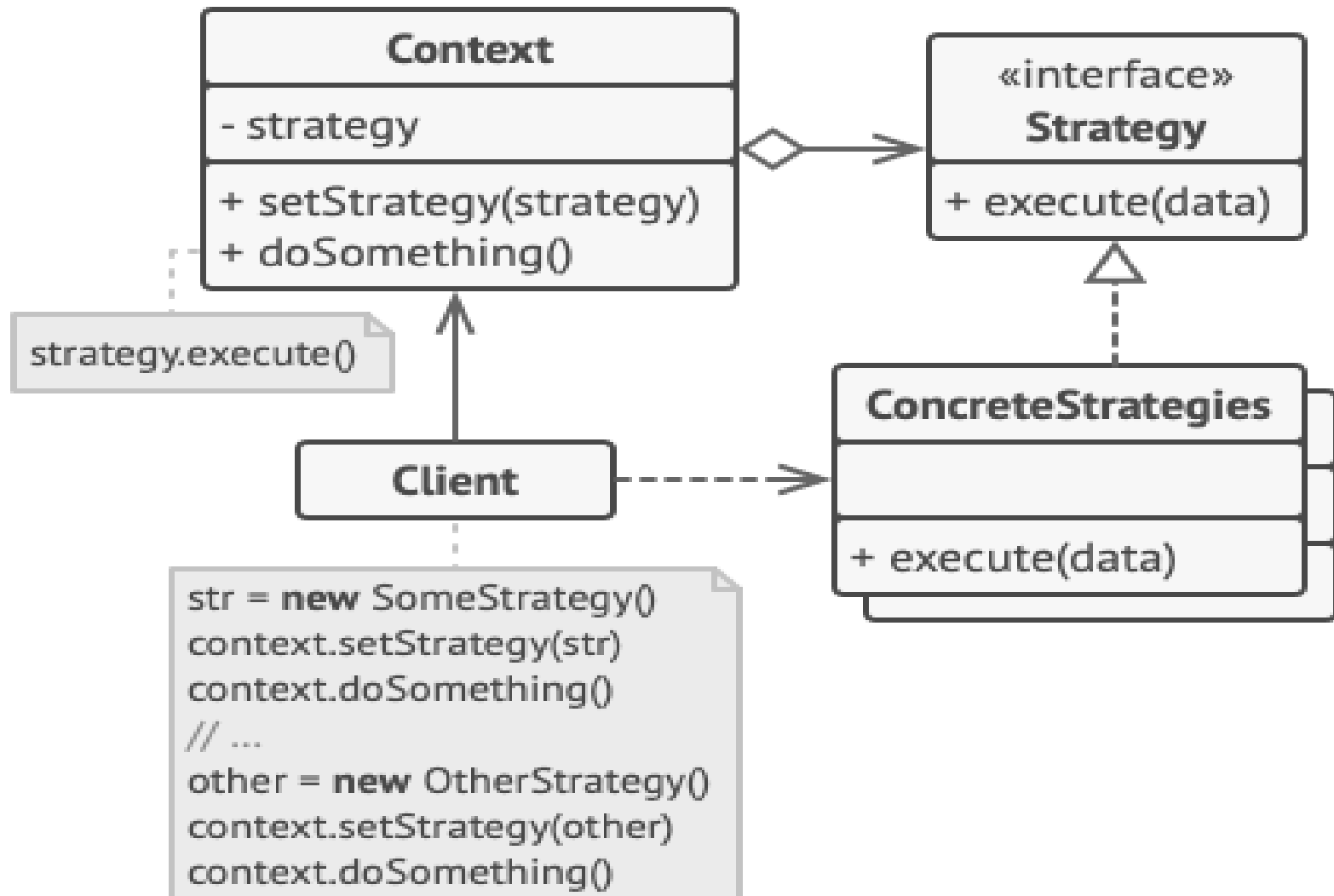
 Định nghĩa (GoF)

- Strategy Pattern định nghĩa một họ các thuật toán, đóng gói từng thuật toán, và cho phép chúng thay đổi độc lập với client sử dụng nó.

 Ý tưởng chính:

- Tách hành vi thay đổi ra thành các class riêng
- Client không cần biết chi tiết cài đặt
- Có thể thay đổi strategy tại runtime

❁ 4. Cấu trúc Strategy Pattern





5. Áp dụng Strategy Pattern – Code (1/2)

5.1 Interface Strategy

```
public interface IShippingStrategy
{
    decimal CalculateShippingFee(Order order);
}
```

5.2 Các Concrete Strategy

```
public class StandardShipping : IShippingStrategy {
    public decimal CalculateShippingFee(Order order) => 20000;
}

public class ExpressShipping : IShippingStrategy {
    public decimal CalculateShippingFee(Order order) => 50000;
}

public class SameDayShipping : IShippingStrategy {
    public decimal CalculateShippingFee(Order order) => 80000;
}
```



5. Áp dụng Strategy Pattern – Code (2/2)

5.3 Context – Order Service

```
public class OrderService {  
    private IShippingStrategy _shippingStrategy;  
    public OrderService(IShippingStrategy shippingStrategy) {  
        _shippingStrategy = shippingStrategy;  
    }  
    public decimal CalculateTotal(Order order) {  
        return order.GetTotal() +  
        _shippingStrategy.CalculateShippingFee(order);  
    }  
}
```

5.4 Sử dụng tại runtime

```
Order order = new Order();  
IShippingStrategy strategy = new ExpressShipping();  
OrderService service = new OrderService(strategy);  
decimal total = service.CalculateTotal(order);
```

✂ Thay đổi thuật toán mà không sửa OrderService



6. Áp dụng Strategy Pattern – Giảm giá (1/2)

6.1 Interface

```
public interface IDiscountStrategy {  
    decimal ApplyDiscount(decimal total);  
}
```

6.2 Concrete Strategies

```
public class NoDiscount : IDiscountStrategy {  
    public decimal ApplyDiscount(decimal total) => total;  
}  
  
public class PercentageDiscount : IDiscountStrategy {  
    private readonly decimal _percent;  
    public PercentageDiscount(decimal percent) {  
        _percent = percent;  
    }  
    public decimal ApplyDiscount(decimal total) => total * (1 - _percent);  
}
```




6. Áp dụng Strategy Pattern – Kết hợp (2/2)

6.3 Kết hợp nhiều Strategy

```
public class CheckoutService {
    private readonly IShippingStrategy _shipping;
    private readonly IDiscountStrategy _discount;

    public CheckoutService(IShippingStrategy shipping,
        IDiscountStrategy discount) {
        _shipping = shipping;
        _discount = discount;
    }

    public decimal Checkout(Order order) {
        decimal total = order.GetTotal();
        total += _shipping.CalculateShippingFee(order); // Strategy 1
        total = _discount.ApplyDiscount(total);          // Strategy 2
        return total;
    }
}
```



7. Strategy Pattern & SOLID

SOLID	Liên hệ
SRP	Mỗi strategy 1 nhiệm vụ
OCP	Thêm thuật toán không sửa code cũ
LSP	Các strategy thay thế cho nhau
ISP	Interface nhỏ, đúng hành vi
DIP	Context phụ thuộc abstraction



8. Unit Test Strategy

```
[Test]
public void ExpressShipping_Should_Return_50000()
{
    IShippingStrategy shipping = new ExpressShipping();
    var fee = shipping.CalculateShippingFee(new Order());

    Assert.AreEqual(50000, fee);
}
```

//  Không cần mock phức tạp

VS 9. Strategy Pattern vs If–Else

Tiêu chí	If–Else	Strategy
Mở rộng	Khó	Dễ
Test	Khó	Dễ
Bảo trì	Dễ lỗi	Ổn định
SOLID	Vi phạm	Tuân thủ



10. Bài tập thực hành

Bài 1:

- Áp dụng Strategy Pattern cho:
- Payment: Cash, CreditCard, MoMo

Bài 2:

- Cho phép thay đổi strategy tại runtime

Bài 3:

- So sánh:
- Strategy Pattern vs Factory Pattern trong bài toán này

11. Tổng kết

- ✓ Strategy Pattern giúp loại bỏ if–else
- ✓ Áp dụng rất nhiều trong hệ thống thực tế
- ✓ Là nền tảng cho Clean Architecture
- ✓ Cực kỳ phù hợp cho Order Management